

Daytona · L0—L5 多视角深度架构图

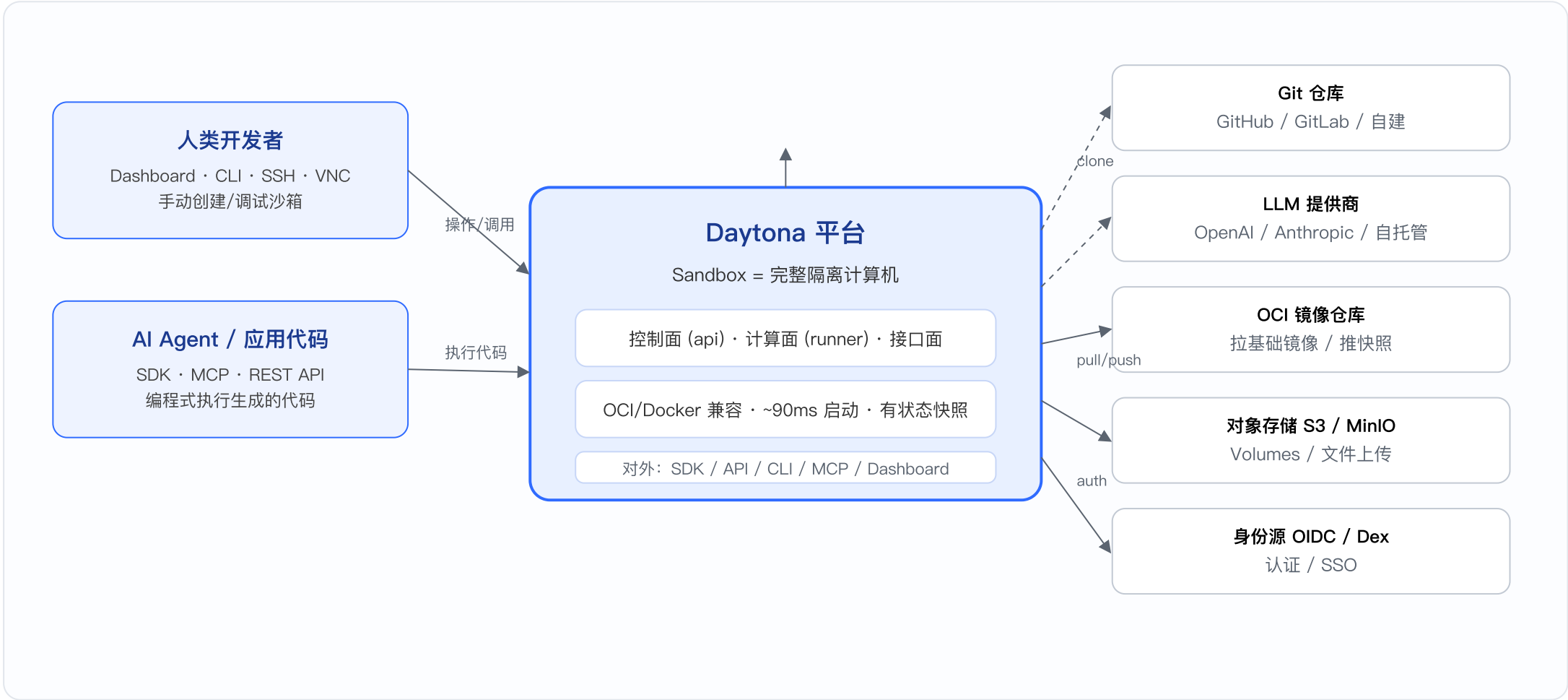
六个不同视角，由浅入深：系统上下文 → 部署拓扑 → 控制面内部 → 编排调度流 → 计算面内部 → 沙箱运行时内核

L0—L5 不是分层栈，而是 六张视角不同的详细架构图，抽象层级由浅到深。每一层都可以独立展开讲解。

- L0 系统上下文
- L1 部署拓扑
- L2 控制面内部
- L3 编排调度流
- L4 计算面内部
- L5 沙箱运行时

L0 · 系统上下文图（最浅：Daytona 在世界里的位置）System Context

最高抽象：只回答「谁在用、依赖什么外部系统」。不画内部模块。

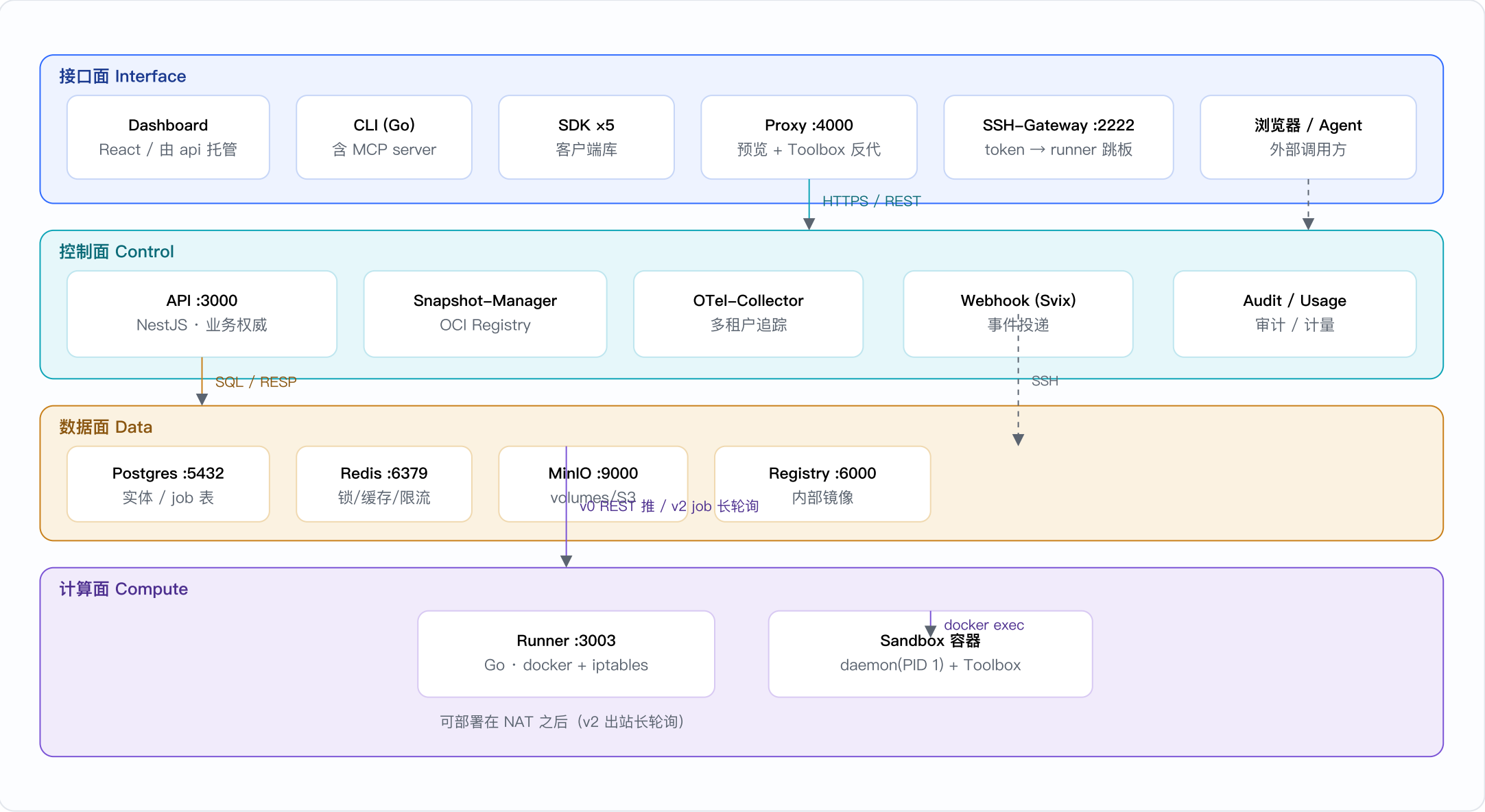


- 这一层只回答三件事：
- **谁用：**人类开发者（可视化/终端）与 AI Agent / 应用代码（程式化）。
 - **平台是什么：**把「隔离计算机」作为托管原语对外暴露，对外统一入口是 SDK/API/CLI/MCP/Dashboard。
 - **依赖谁：**Git（取代码）、LLM（Agent 自管，平台不绑定）、OCI 仓库（镜像/快照）、对象存储（卷）、OIDC（身份）。

L1 · 部署 / 容器拓扑图（运行时可部署构件与协议）

Container / Deployment

把平台拆成可独立部署/运行的进程（容器/二进制），并标注它们之间的通信协议与端口。



部署要点：

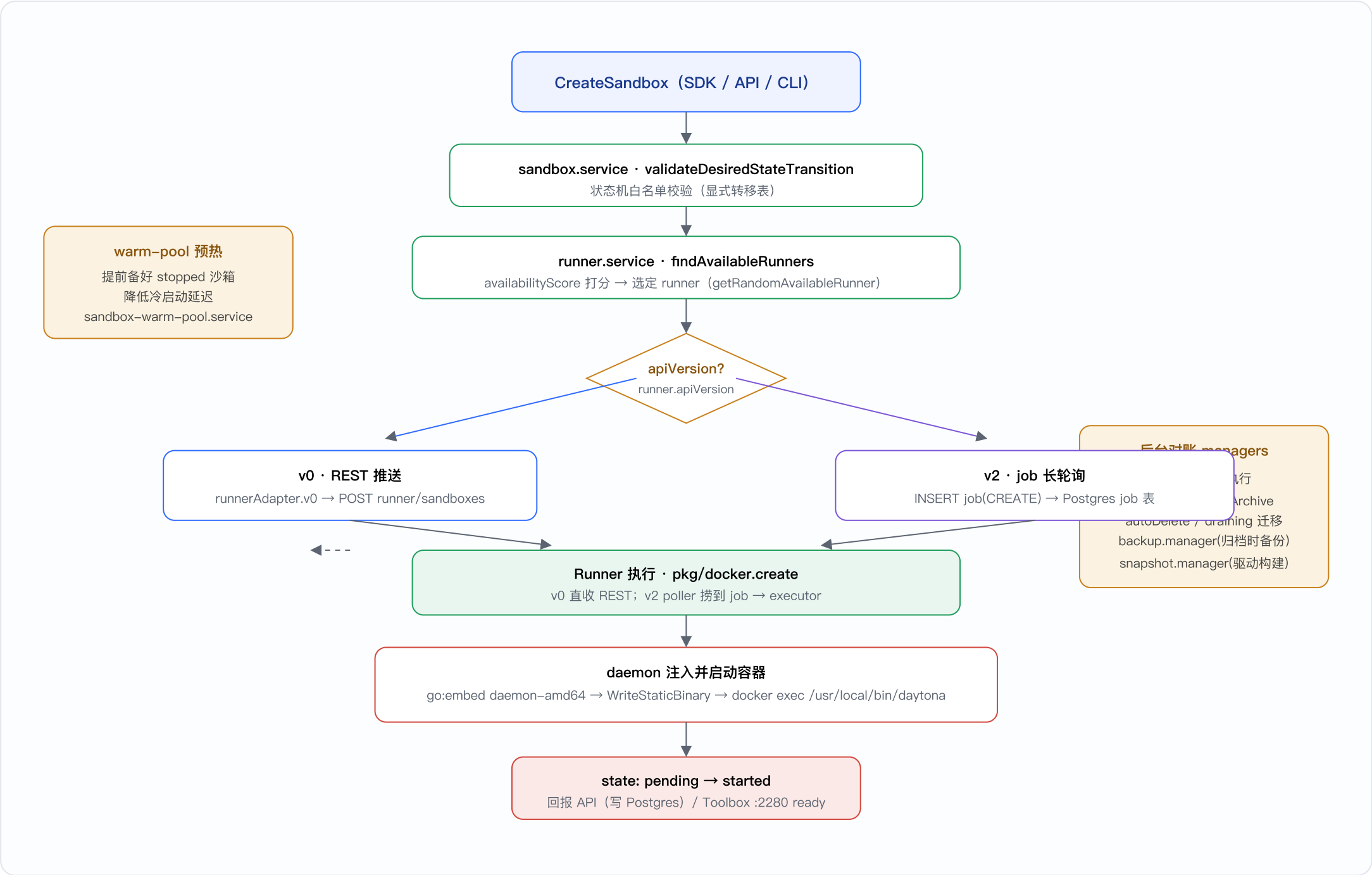
- 自托管参考为 docker-compose (15+ 服务)；Runner 还能 deb/systemd 裸机部署。
- 控制面与计算面解耦：v2 模式下 Runner 只需出站长轮询 POST /jobs/poll，可置于 NAT 之后。
- Proxy / SSH-Gateway 是用户到沙箱的两条独立入口；Proxy 同时承载预览与 Toolbox 反向代理。

L2 · 控制面内部模块图（apps/api 的内脏）

Control Plane Internals

展开 L1 里的 API 进程：NestJS 模块边界、核心域（sandbox）、跨切面模块，以及它与数据存储的关系。



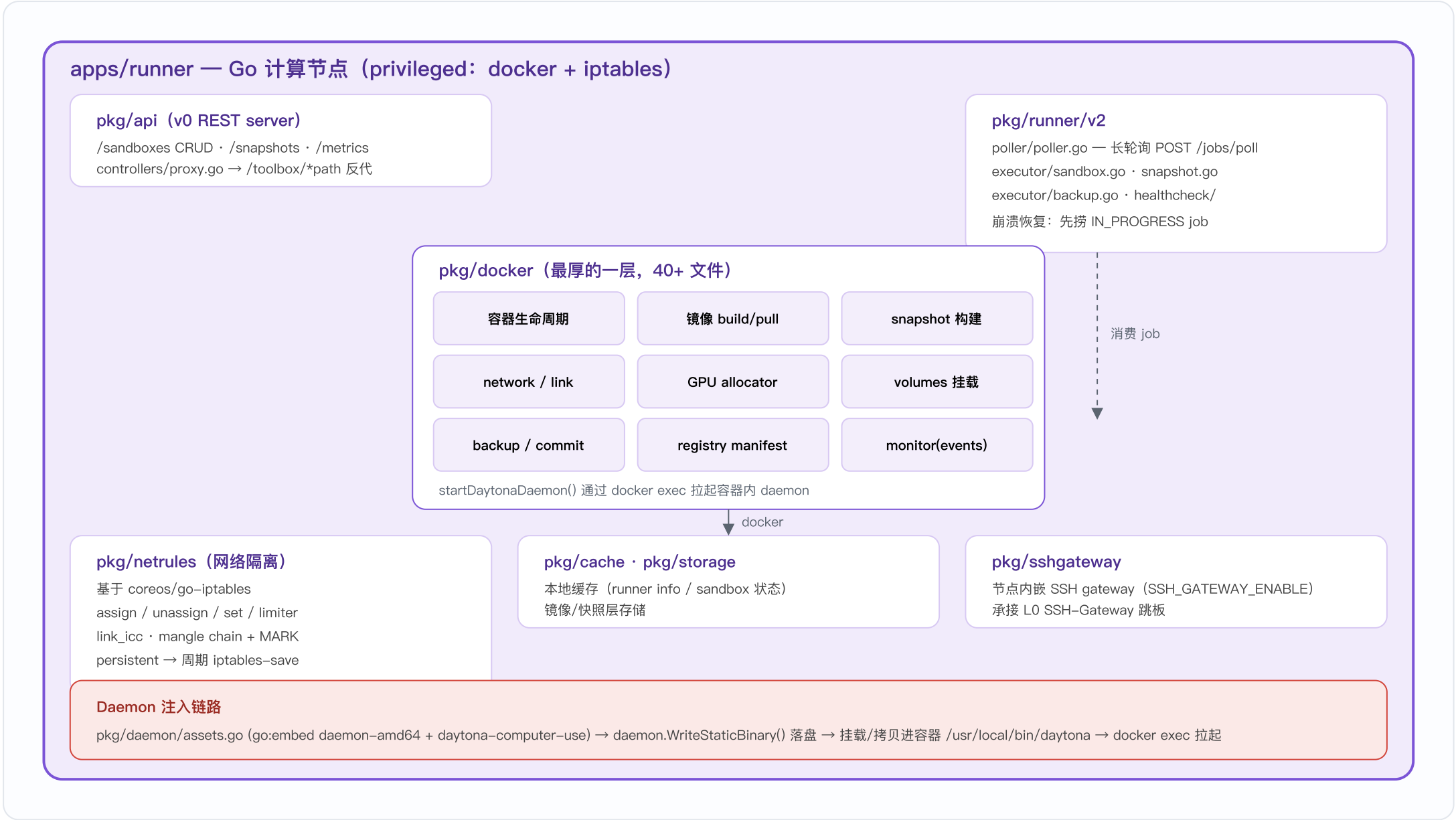


- 编排调度的要点：
- 状态机先行：任何操作先过 `validateDesiredStateTransition` 的显式转移白名单，非法转移直接拒绝。
 - 调度在控制面：runner.service 用 `availabilityScore` 打分选 runner，与执行解耦。
 - v0/v2 是同一意图的两种投递：v0 控制面直推 REST；v2 写 job 表、Runner 出站长轮询。后者让 Runner 可置于 NAT 后。
 - 后台对账独立运转：warm-pool 预热、autoStop/Archive/Delete、归档时 backup、快照构建，都由 @Cron managers 驱动，与请求路径正交。

L4 · 计算面内部模块图（apps/runner 的内脏）

Compute Plane Internals

展开 L1 的 Runner 进程：Go 包边界、docker/netrules/v2 三层职责，以及 daemon 注入链路。



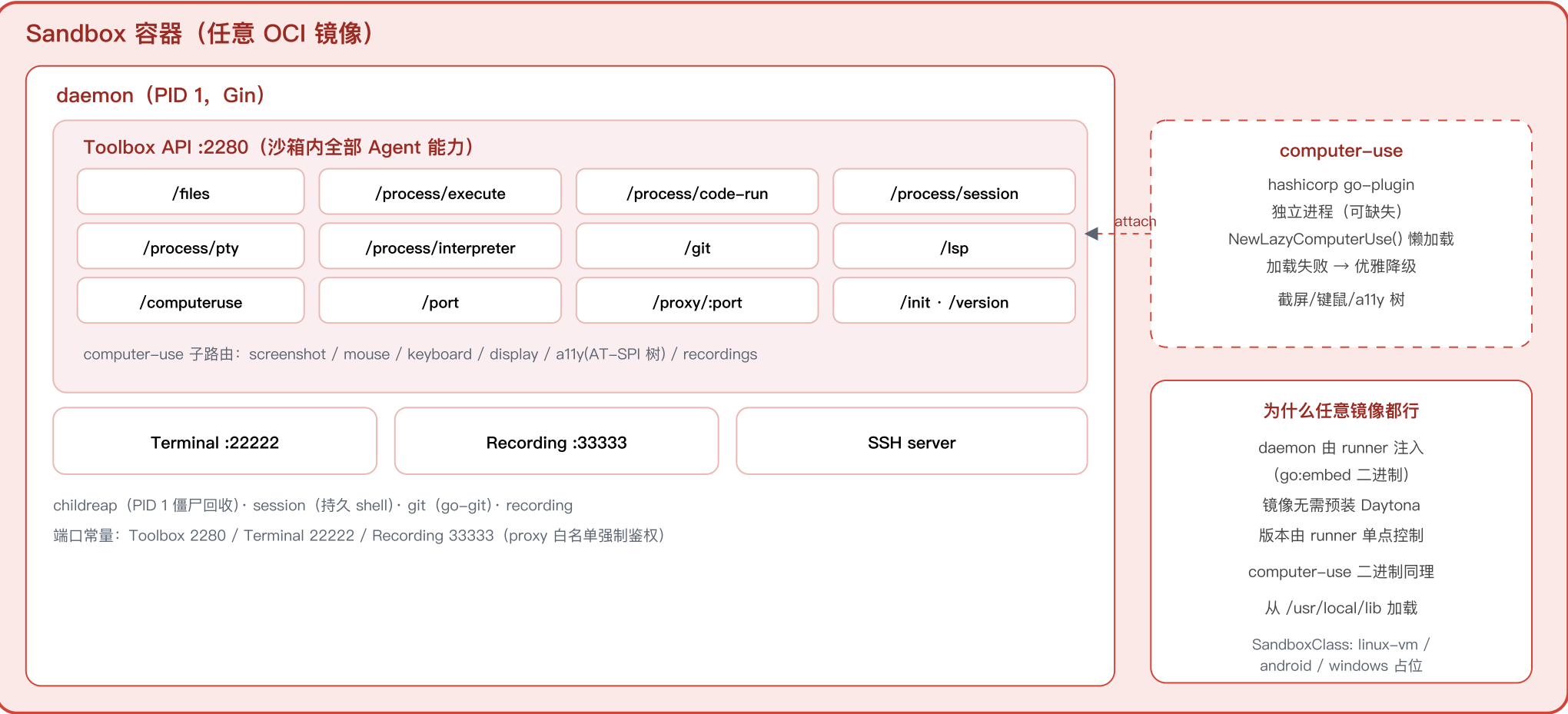
计算面内部关键事实：

- **pkg/docker 是绝对核心**：容器生命周期、镜像、快照、网络、GPU、卷、备份、监控全部在此；Runner 的实际计算后端目前只有 Docker。
- **v0/v2 在计算面都有对应实现**：v0 是 pkg/api 的 REST 面；v2 是 pkg/runner/v2 的 poller+executor，Runner 主动拉 job。
- **网络隔离在节点侧**：pkg/netrules 用 iptables 实现 networkBlockAll/AllowList，由 API 经 UPDATE_SANDBOX_NETWORK_SETTINGS job 下发。
- **daemon 不在镜像里**：runner 用 go:embed 内嵌二进制，启动时落盘注入——任意 OCI 镜像即可当沙箱。

L5 · 沙箱运行时 / 内核细节图（容器内到底发生了什么）

Sandbox Runtime / Kernel

最深的视角：展开 L4 拉起的那个容器——daemon(PID 1)、Toolbox 全部路由、computer-use 插件，以及用户请求如何一路代理进来。



沙箱运行时最深的细节：

- daemon 是 PID 1，在沙箱内同时起 4 个 server：Toolbox(:2280) / Terminal(:22222) / Recording(:33333) / SSH，并负责 PID 1 的僵尸回收。
- Toolbox 是 Agent 能力的全集：文件系统、进程执行、持久会话、PTY、有状态解释器、Git、LSP、Computer-use、端口、沙箱内端口转发——全部走 :2280。
- computer-use 是唯一运行时插件：hashicorp go-plugin 独立进程，懒加载 + 优雅降级，缺失也不影响其它能力。
- 请求绝不直连：SDK → Proxy(:4000) → Runner(:3003)/toolbox/* → daemon(:2280)，三层代理 + 鉴权。